

Incident Memory Demo Project

Technical Overview and Verification Guide

For Software Engineers and SREs

Opening summary

This document describes the **Incident Memory** demo: a small production-oriented prototype that combines semantic incident retrieval (Qdrant), large language models (LLMs), and a persistent learning layer (Hindsight) to reduce repeated incidents and accelerate on-call resolution. It is written for engineers and SREs who will verify, extend, or integrate the demo into their toolchains.

The project demonstrates an end-to-end flow: ingest incidents, index them semantically, surface similar incidents during search, augment LLM prompts with recalled experiences, and persist lessons learned so the system improves over time.

What the project is and its goals

The Incident Memory demo implements **semantic incident memory** plus **long-term learning**. Its goals are:

- Reduce knowledge loss across rotations by capturing incident experiences.
- Surface prior resolutions and lessons to on-call engineers via semantic search and LLM suggestions.
- Persist distilled experiences so the system learns from resolved incidents and improves future suggestions.

One-sentence status: core app plus Hindsight integration (PHASES 1–10 complete; verification ongoing).

Background and motivation

On-call teams repeatedly face the same classes of incidents. Knowledge is often trapped in postmortems, chat logs, or the memory of a few engineers. This leads to repeated toil, slow incident resolution, and incomplete postmortems. The demo addresses three problems:

1. On-call knowledge loss across rotations.
2. Repeated incidents with similar root causes.
3. Slow or incomplete postmortems that fail to feed back into tooling.

Combining **Qdrant** (semantic similarity), **Hindsight** (persistent learning), and **LLMs** (suggestions and synthesis) is useful because:

- Qdrant finds semantically similar incidents even when wording differs.

- Hindsight stores structured experiences and reflections for long-term recall.
- LLMs synthesize context, propose fixes, and incorporate memories to produce actionable suggestions.

Architecture and components (diagram-friendly narrative)

High-level components:

- **Postgres** — canonical truth store for incident records and metadata.
- **Qdrant** — vector index for semantic retrieval of incident text.
- **LLM** (Gemini/OpenAI) — generates suggestions and summaries.
- **Hindsight** — persistent experience bank for retain/recall/reflect.
- **Web app / API** — app.py and supporting modules that orchestrate flows.

Map of files to responsibilities

- **app.py** — HTTP API endpoints: ingest, search, resolve, and wiring to Hindsight and Qdrant.
- **suggest_fix.py** — builds LLM prompts, calls LLM, and returns suggested fixes.
- **new_ingest.py** — ingestion pipeline: write to Postgres and index to Qdrant.
- **query_incidents.py** — query helpers for Postgres and Qdrant.
- **resolve_incident.py** — resolution flow and finalization logic.
- **templates/index.html** — simple UI for manual testing and viewing suggestions.
- **backend/hindsight/*** — Hindsight integration: client wrapper, schemas, retain/recall/reflect helpers, and tests.

End-to-end data flow

1. **Ingest** (new_ingest.py): incident created in Postgres, text indexed in Qdrant.
2. **Search** (query_incidents.py): user query triggers Qdrant semantic search.
3. **LLM Suggestion** (suggest_fix.py): prompt assembled from current incident, Qdrant results, and Hindsight memories; LLM returns suggestions.
4. **Resolve** (resolve_incident.py & app.py): engineer marks incident resolved; resolution and root cause are recorded.
5. **Retain** (backend/hindsight/retain.py): resolved incident converted to an IncidentExperience and sent to Hindsight for long-term storage.

Hindsight integration — implementation details

Integration lives under backend/hindsight/.

Client wrapper

backend/hindsight/client.py exposes:

- **HindsightConfig** — configuration dataclass reading connection parameters.
- **HindsightClientWrapper** — wrapper that discovers available SDKs/adapters and provides a unified API.
- **available()** — checks whether a Hindsight SDK/adaptor is importable and reachable.
- **connect()** — establishes a client instance using HINDSIGHT_API_URL, HINDSIGHT_API_KEY, HINDSIGHT_BANK_ID (server-side only).

Schema

backend/hindsight/schemas.py defines **IncidentExperience** and a `to_dict()` method that serializes the experience for the Hindsight API.

Retain helpers

- backend/hindsight/retain.py — helper functions to call the client.retain API.
- backend/hindsight/retain_historical.py — batch/historical retain with deduplication logic keyed by incident_id.

Recall and reflect helpers

- backend/hindsight/recall.py — `recall_memories(client, text, top_k)` wraps `client.recall()` and returns memory objects.
- backend/hindsight/reflect.py — `reflect_memories(client, memories)` calls `client.reflect()` to produce a short reflection when enough memories exist.

Environment variables (server-side only)

The server reads:

```
HINDSIGHT_API_URL  
HINDSIGHT_API_KEY  
HINDSIGHT_BANK_ID
```

Exact retain call chain

The live retain path is:

1. HTTP POST `/api/resolve` in `app.py`.
2. Build an `IncidentExperience` instance.
3. Call `retain_experience(client, exp)`.
4. `retain_experience` calls `client.retain(exp.to_dict())`.

Retained fields

The payload retains these fields (exact names):

- `incident_id`
- `title`
- `service`
- `severity`
- `status`
- `description`
- `root_cause`
- `resolution`
- `outcome` (default: `'resolved'`)
- `lesson` (typically `null` unless provided)

```
{
  "incident_id": "INC-2026-0001",
  "title": "Database connection exhaustion",
  "service": "auth-service",
  "severity": "P1",
  "status": "resolved",
  "description": "Connections exhausted under load",
  "root_cause": "Connection leak in auth service",
  "resolution": "Restarted service and patched connection pool",
  "outcome": "resolved",
  "lesson": null
}
```

LLM prompt and how Hindsight augments it

`suggest_fix.py` builds the prompt by concatenating:

1. **CURRENT INCIDENT** — title, description, severity, recent logs.
2. **QDRANT RESULTS** — top semantic matches with short excerpts.
3. **HINDSIGHT MEMORIES** — recalled experiences from `client.recall()`.
4. **OPTIONAL HINDSIGHT REFLECTION** — included only when `client.reflect()` returns `{'ok': True}`.

Recall path Search endpoint flow:

- `/api/search` calls `recall_memories(client, text, top_k)`.
- `recall_memories` calls `client.recall()`.
- Returned memories are inserted into the LLM prompt as a "Relevant past incidents" section.

Reflect path If ≥ 2 memories are returned, the code calls `client.reflect()`. Only when the reflection returns `{'ok': True}` is the reflection text appended to the prompt as "Reflection: ...".

```
--- CURRENT INCIDENT ---
Title: Database connection exhaustion
Description: Connections exhausted under load...

--- RELEVANT PAST INCIDENTS (HINDSIGHT) ---
- INC-2025-042: Connection leak in auth-service; resolution: patched pool; lesson:
  ensure pool close on error.
- INC-2024-311: DB max connections reached during deploy; resolution: rollback +
  increase pool.

--- REFLECTION ---
Reflection: Multiple incidents point to connection lifecycle issues in auth-service.
```

Verification & testing performed so far

- Files inspected statically: `app.py`, `suggest_fix.py`, `backend/hindsight/*`, `new-ingest.py`, `query_incidents.py`, `resolve_incident.py`, `templates/index.html`.
- Static traces confirm the retain call chain and prompt assembly.
- Test helper added: `backend/hindsight/test_client.py` to exercise available/`connect/retain/recall/reflect` paths against a local or mocked Hindsight endpoint.

Manual test plan (run locally)

Start server:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
```

```
curl -X POST http://localhost:8000/api/ingest \
-H "Content-Type: application/json" \
-d '{"incident_id":"INC-TEST-1","title":"DB connection exhaustion","service":"auth-
service","description":"..."}'
```

```
curl -X POST http://localhost:8000/api/search \
-H "Content-Type: application/json" \
-d '{"query":"connection exhaustion auth-service"}'
```

```
curl -X POST http://localhost:8000/api/resolve \
-H "Content-Type: application/json" \
-d '{"incident_id":"INC-TEST-1","root_cause":"Connection leak","resolution":"Patched pool","status":"resolved"}'
```

PowerShell equivalents use `Invoke-RestMethod` with `-Body JSON`.

Notes Runtime attempts to call a real Hindsight service are blocked in this environment; reproduce locally by setting:

```
export HINDSIGHT_API_URL="https://hindsight.example"
export HINDSIGHT_API_KEY="secret"
export HINDSIGHT_BANK_ID="bank-123"
```

Then run `python backend/hindsight/test_client.py`.

Known gaps, risks, and recommended next steps

- **HIGH:** Duplicate retention gap — `/api/resolve` lacks dedupe; `retain_historical.py` has dedupe but live path does not. Fix: call dedupe logic from live retain path or centralize retain with dedupe.
- **MEDIUM-HIGH:** Outcome granularity missing — only `outcome='resolved'` stored. Add richer outcome taxonomy (mitigated, monitoring, rollback).
- **MEDIUM:** Reflection depends on SDK presence; if SDK not available, reflect is skipped. Provide a fallback local reflection or a mock adapter.
- **MEDIUM:** Tests missing — add unit and integration tests that mock Hindsight client and Qdrant responses.
- **LOW:** Documentation could include example SDK adapter instructions and a sample Hindsight bank schema.

Illustrative example / mini-case study

Before learning: Incident "Database connection exhaustion" arrives. LLM sees only the current incident and Qdrant matches and returns a generic recommendation: "Restart DB and increase pool."

Resolve + retain: Engineer records root cause "Connection leak in auth service" and resolution "Patched connection handling; added metrics." The system retains an `IncidentExperience` with those fields.

After learning: A new similar incident arrives. During `/api/search`, Hindsight recall returns the prior experience. The LLM prompt now includes the prior root cause and resolution, biasing the LLM toward recommending the specific patch and metrics to monitor. Expected prompt

excerpt (see earlier prompt excerpt) and LLM influence: more targeted remediation steps and a suggested postmortem lesson.

How to run and reproduce (copyable commands)

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export HINDSIGHT_API_URL="https://hindsight.example"
export HINDSIGHT_API_KEY="secret"
export HINDSIGHT_BANK_ID="bank-123"
python app.py
python backend/hindsight/test_client.py
# Then use the curl commands in the TEST 1-3 section
```

Conclusion and call to action

This demo shows how semantic retrieval plus persistent experience storage can materially reduce repeated incidents and improve on-call outcomes. Next priorities: add dedupe to the live retain path, capture richer outcome metadata, and add unit/integration tests that mock Hindsight and Qdrant.

If you want to help: run the verification steps locally, implement dedupe in `/api/resolve`, add outcome capture in the resolve flow, and contribute tests and SDK adapter docs.

What I did for this hackathon

I inspected and modified files under `backend/hindsight/*`, updated `suggest_fix.py` to include Hindsight memories in prompts, wired retain calls in `app.py`, adjusted `templates/index.html` for testing, and added `README.md` documentation and `backend/hindsight/test_client.py` test helper.